

**Data Wrangling** 🤠

# **Math for Naive Bayes**

Lesson 16

# Overview

- Our model will look at some text and assign it a label:
  - "this movie was boring and dumb" → negative
- Some words are strong evidence for a label while some words are just noise
- How do we take these words and turn them into **prediction scores** or numbers?
- We'll discuss all the math you'll need! 🧮

# Your Training Data

- Every row is one example with one `label` and some `content`
- Your `train.csv` will look something like this:

| <code>label</code> | <code>content</code>                 |
|--------------------|--------------------------------------|
| positive           | wonderful acting and wonderful story |
| positive           | wonderful movie loved it             |
| negative           | boring acting and dull               |
| negative           | terrible movie hated it              |

# Training Steps

Before it can predict anything, your classifier reads the training data and counts four things:

| Symbol      | Meaning                                                 |
|-------------|---------------------------------------------------------|
| $N$         | total number of training examples                       |
| $N_C$       | number of examples with label $C$                       |
| $N_w$       | number of examples that contain word $w$ (any label)    |
| $N_{(C,w)}$ | number of examples with label $C$ that contain word $w$ |

# Counting Examples: N and N\_C

From our 4-row table:

- **N** = 4 (total examples)
- **N\_positive** = 2
- **N\_negative** = 2

# Counting Words: $N_w$

- Remember we are using the **bag of words** model, so for each example, we only care about *which* words appear, not how many times
- "wonderful acting and wonderful story" →  
{wonderful, acting, and, story}

Counting how many examples (regardless of label) contain each word.

# Word counts:

| word      | N_w |
|-----------|-----|
| wonderful | 2   |
| acting    | 2   |
| movie     | 2   |
| and       | 2   |
| story     | 1   |
| loved     | 1   |

# Word counts cont'd:

| word     | N_w |
|----------|-----|
| it       | 2   |
| boring   | 1   |
| dull     | 1   |
| terrible | 1   |
| hated    | 1   |



# Counting Words *Per Label*: $N_{\text{C,w}}$

Now we count and split by label. Keep track of how many **positive** examples contain each word, and how many **negative** examples contain each word:

**Positive words:**

| word      | $N_{\text{positive, w}}$ |
|-----------|--------------------------|
| wonderful | 2                        |
| acting    | 1                        |
| movie     | 1                        |

## Positive words cont'd:

| word     | $N_{\text{positive}}(w)$ |
|----------|--------------------------|
| and      | 1                        |
| story    | 1                        |
| loved    | 1                        |
| it       | 1                        |
| boring   | 0                        |
| dull     | 0                        |
| terrible | 0                        |
| hated    | 0                        |

## Negative words:

| word      | $N_{\text{negative}}(w)$ |
|-----------|--------------------------|
| wonderful | 0                        |
| acting    | 1                        |
| movie     | 1                        |
| and       | 1                        |
| story     | 0                        |
| loved     | 0                        |
| it        | 1                        |

## Negative words cont'd:

| word     | $N_{\text{(negative, w)}}$ |
|----------|----------------------------|
| boring   | 1                          |
| dull     | 1                          |
| terrible | 1                          |
| hated    | 1                          |

# $N_w$ compared to $N(C, w)$

| word      | $N(\text{positive}, w)$ | $N(\text{negative}, w)$ |
|-----------|-------------------------|-------------------------|
| wonderful | 2                       | 0                       |
| acting    | 1                       | 1                       |
| movie     | 1                       | 1                       |
| and       | 1                       | 1                       |
| story     | 1                       | 0                       |

| word     | N_(positive, w) | N_(negative, w) |
|----------|-----------------|-----------------|
| loved    | 1               | 0               |
| it       | 1               | 1               |
| boring   | 0               | 1               |
| dull     | 0               | 1               |
| terrible | 0               | 1               |
| hated    | 0               | 1               |

# Training is Done!

- Everything we compute regarding prediction will come from these 4 counts:  $N$ ,  $N_C$ ,  $N_w$ ,  $N(C,w)$
- Now, as we see new text examples, we ask:
  - 1. How common is each label, on its own?**
  - 2. How well do the the words in this text (e.g. review) fit each label?**

# 1. How common is each label?

Before even reading the words in a new review, consider how likely each label  $C$  is just based on how often it shows up in training data. This is the **log-prior**.

$$\text{log-prior}(C) = \ln( N_C \div N )$$

For our dataset:

- $\text{log-prior}(\text{positive}) = \ln(2 \div 4) = \ln(0.5) \approx \mathbf{-0.693}$
- $\text{log-prior}(\text{negative}) = \ln(2 \div 4) = \ln(0.5) \approx \mathbf{-0.693}$



# Why take the log?

- The numbers we are computing before applying the log are actually probabilities.
- Why not just multiply plain probabilities? Because multiplying many small fractions together produces **extremely** tiny numbers!
- Logs are also **monotonic**: bigger input always gives a bigger output so comparing log-scores directly is exactly as valid as comparing the original probabilities
-  **You cannot take the log of zero!**

## 2. How well does a word fit a label?

If we are considering label  $C$ , how much does the presence of a single word  $w$  in the review support that? The **log-likelihood** tell us this.

$$\text{log-likelihood}(w, C) = \ln( N_{(C,w)} \div N_C )$$

For our dataset considering label positive ( $N_{\text{positive}} = 2$ ):

- **wonderful**:  $N_{(\text{positive}, \text{wonderful})} = 2 \rightarrow \ln(2 \div 2) = \ln(1) = 0$
- **acting**:  $N_{(\text{positive}, \text{acting})} = 1 \rightarrow \ln(1 \div 2) \approx -0.693$

Notice that **wonderful** under **positive** is exactly 0! 🤔

## 2. What if we are missing data?

What if a word never showed up with a given label C at all?

| Case | Condition                                | Formula                     |
|------|------------------------------------------|-----------------------------|
| 1    | word appeared with label C at least once | $\ln( N_{(C,w)} \div N_C )$ |
| 2    | word appeared, but never with label C    | $\ln( N_w \div N )$         |
| 3    | word never appeared in training at all   | $\ln( 1 \div N )$           |

# Example: Choosing a Fallback Case

`wonderful` never appeared in any **negative** example:  
 $N_{\text{(negative,wonderful)}} = 0$ .

- Case 1 doesn't apply (0 examples)
- Check case 2: Did `wonderful` appear in training at all? Yes, we see that  $N_{\text{wonderful}} = 2$ . **So we can use case 2.**

# Example: Choosing a Fallback Case

$\text{log-likelihood}(\text{wonderful}, \text{negative}) = \ln( N_w \div N ) =$   
 $\ln(2 \div 4) = \ln(0.5) \approx \mathbf{-0.693}$

Compare to `wonderful` under **positive**, which scored exactly **0**! Do you think the word `wonderful` supports the label `negative` or `positive`?

# Concept Check

Using the  $N_{(C,w)}$  table from a few slides ago, which case (1, 2, or 3) applies to each, and why?

- **boring** under label **positive**
- **hated** under label **negative**
- **amazing** under label **positive** (a word that never appeared anywhere in training)

### 3. Scoring a Whole Piece of Text

To score an entire piece of text against a label, combine the log-prior with the log-likelihood of **every unique word** in the text:

**$\text{score}(\text{text}, C) = \text{log-prior}(C) + \text{sum of log-likelihood}(w, C) \text{ for each unique word } w \text{ in text}$**

- More words that fit well will mean higher (less negative) score for that label
- We compute a score for **every single label** that exists

## 4. Making a Prediction

- Let us combine all of these steps to predict the label for new, unlabeled text: **"wonderful acting movie"**



# Example: Scoring under **positive**

| Term                                | Value  |
|-------------------------------------|--------|
| log-prior(positive)                 | -0.693 |
| log-likelihood(wonderful, positive) | 0      |
| log-likelihood(acting, positive)    | -0.693 |
| log-likelihood(movie, positive)     | -0.693 |

$$\text{score}(\text{text}, \text{positive}) = -0.693 + 0 + (-0.693) + (-0.693) = \mathbf{-2.079}$$

# Example: Scoring under **negative**

| Term                                | Value                    |
|-------------------------------------|--------------------------|
| log-prior(negative)                 | -0.693                   |
| log-likelihood(wonderful, negative) | -0.693 (fallback case 2) |
| log-likelihood(acting, negative)    | -0.693                   |
| log-likelihood(movie, negative)     | -0.693                   |

$$\text{score}(\text{text}, \text{negative}) = -0.693 + (-0.693) + (-0.693) + (-0.693) =$$

**-2.772**

## 4. Making a Prediction

- Compute **score(text, C)** for **every possible label**
- Predict whichever label has the **highest** score
- If there is a tie, predict whichever label comes first **alphabetically**

For our example:

- $\text{score}(\text{positive}) = -2.079 > \text{score}(\text{negative}) = -2.772$

**Prediction: positive** 

# 5. Evaluating on Test Data

Once training is done, run the prediction process on **every row of your test file**:

1. Predict a label for the text
2. Compare the prediction to the row's true label
3. Count how many predictions were correct

**accuracy = (number of correct predictions) ÷ (total number of test examples)**

# Formula Cheat Sheet

| Step                    | Formula                                                    |
|-------------------------|------------------------------------------------------------|
| Log-prior               | $\ln( N_C \div N )$                                        |
| Log-likelihood (case 1) | $\ln( N_{(C,w)} \div N_C )$                                |
| Log-likelihood (case 2) | $\ln( N_w \div N )$                                        |
| Log-likelihood (case 3) | $\ln( 1 \div N )$                                          |
| Score                   | $\text{log-prior}(C) + \text{sum of log-likelihood}(w, C)$ |
| Prediction              | label C with the highest score(text, C)                    |
| Accuracy                | correct predictions $\div$ total predictions               |

# Summary

- Training a Naive Bayes classifier means counting: **N**, **N\_C**, **N\_w**, **N\_(C,w)**
- **Log-prior** measures how common a label is on its own; **log-likelihood** measures how well one word fits a label
- Choose a *fallback case* for unseen or rare words
- **Score** = log-prior + the sum of log-likelihoods for every unique word in the text
- **Predict** = whichever label scores highest
- **Accuracy** = correct ÷ total on the test set